

Contents

1	詳細設計 index (メインシステム)	1
1.1	0. 文書情報	1
1.2	1. 詳細設計方針	2
1.2.1	1.1 目的	2
1.2.2	1.2 対象システム	2
1.2.3	1.3 スcope	2
1.2.4	1.4 関連ドキュメント	2
1.3	2. システム全体構成	2
1.3.1	2.1 全体構成図	2
1.3.2	2.2 コンポーネント詳細責務	3
1.3.3	2.3 リクエストフロー	4
1.3.4	2.4 環境構成	5
1.3.5	2.5 デプロイ単位	6
1.4	3. ディレクトリ構成・モジュール構成	7
1.4.1	3.1 リポジトリレイアウト	7
1.4.2	3.2 レイヤ構成	7
1.4.3	3.3 主要モジュール	8
1.4.4	3.4 共通ライブラリ	11
1.4.5	3.5 設定ファイル	11
1.5	4. 共通処理方針	12
1.5.1	4.1 エラー設計 (参照のみ)	12
1.5.2	4.2 セキュリティ詳細設計 (参照のみ)	12
1.5.3	4.3 ログ設計概要	12
1.5.4	4.4 監視・アラート	12
1.5.5	4.5 実装ガイドライン	12
1.6	5. 詳細設計ファイル一覧	13
1.7	6. 実装順序	14
1.8	7. 非機能設計サマリ	14
1.8.1	7.1 性能設計 (§13.1)	14
1.8.2	7.2 可用性設計 (§13.2)	14
1.8.3	7.3 スケーラビリティ (§13.3)	15
1.8.4	7.4 アクセシビリティ・国際化	15
1.8.5	7.5 バックアップ・復旧 (§13.6)	15
1.8.6	7.6 データ保持・削除 (§13.7)	15
1.8.7	7.7 コンプライアンス (§13.X)	15
1.8.8	7.8 インフラコスト構造 (§13.Y)	15
1.9	8. リリース計画	15
1.9.1	8.1 CI/CD パイプライン	15
1.9.2	8.2 ペネトレーション	16
1.9.3	8.3 リリースゲート	16
1.10	9. 未確定事項・確認事項	16

1 詳細設計 index (メインシステム)

1.1 0. 文書情報

項目	内容
文書名	詳細設計 index (メインシステム)
対象システム	FAQ AI ウィジェット SaaS / メインシステム (管理画面 + 公開ウィジェット + エンドユーザー画面)
版数	v1.0
作成日	2026-05-17
ステータス	承認済
入力文書	../01_要件定義/index.md / ../02_基本設計/index.md / ../画面遷移図.html
実装スタック前提	Cloudflare Workers / Pages / D1 / R2 / KV / Queues + TypeScript + Hono + Zod

位置づけ: 本書は実装関連の詳細(モジュール構成 / ロジック / 非機能 / バッチ / ログ / 監視 / テスト /

リリース / 実装ガイドライン)を扱う。画面・API・DB・権限・エラー・セキュリティ・状態遷移は
02_ 基本設計/ 配下の 10 ドキュメント体系を正本とし、本書からは参照のみ行う。

1.2 1. 詳細設計方針

1.2.1 1.1 目的

基本設計書で定義された FAQ AI ウィジェット SaaS メインシステムの設計を、Cloudflare Workers + TypeScript + Hono + Zod を用いた実装に直接落とし込むレベルまで詳細化することを目的とする。設計の各構成要素について、「どこに何を書くか (ファイル配置)」「何をどう書くか (型・関数・テーブル・SQL・JSON スキーマ)」を明示し、実装者が翌日からコード作成に着手できる粒度を達成する。

1.2.2 1.2 対象システム

本書の対象は、FAQ AI ウィジェット SaaS のメインシステムである。具体的には以下を含む。

- ・ **管理画面** (管理者ユーザー admin 向け、SCR-001~025)
- ・ **公開ウィジェット** (エンドユーザー end_user 向け、JavaScript 配信、iframe sandbox)
- ・ **エンドユーザー専用画面** (SCR-027 お問い合わせ再開)
- ・ **内部連携 API** (顧客管理システムとの IF #1~#12 送受信)
- ・ **非同期処理基盤** (cron / Queue / Webhook 受信)

1.2.3 1.3 スcope

リリース優先度 P0 (必須) に該当する全機能要件・非機能要件を対象とする。P1 のうち P0 と密結合する以下については本書で扱う。

- ・ プロジェクト単位 IP 許可リスト機能 (FR-179 / FR-330、FAQ ウィジェット向け / 管理画面 API は対象外)
- ・ AI 品質回帰テスト (FR-059 / FR-342)

1.2.4 1.4 関連ドキュメント

種別	パス	役割
要件	../01_要件定義/index.md	機能・非機能要件の正本
基本設計	../02_基本設計/index.md	アーキテクチャ・データモデル・状態の正本
詳細設計 (本書)	本ファイル	実装に直結する具体仕様
画面設計書	../画面遷移図.html	画面レイアウト、画面目的、権限、入出力、遷移、例外の視覚情報
運用設計	../04_運用設計/index.md	運用手順 / runbook
将来対応	../05_future/index.md	MVP 後の候補

1.3 2. システム全体構成

1.3.1 2.1 全体構成図

```
flowchart TB
    subgraph Client["クライアント層"]
        direction LR
        EU["エンドユーザー <br/>(ブラウザ + 埋込ウィジェット)"]
        AD["管理者ユーザー <br/>(管理画面ブラウザ)"]
    end

    subgraph CF["Cloudflare エッジ"]
        direction TB
        PG["Pages<br/>(管理画面 SPA / 静的アセット)"]
        WJS["Pages<br/>(widget.js 配信)"]
    end
```

```

subgraph Workers["Workers"]
  direction LR
  MW["main-api Worker<br/>(/api/v1/*)"]
  WW["widget-api Worker<br/>(/widget/v1/*)"]
  IW["internal-api Worker<br/>(/internal/admin-integration/v1/*)"]
  WHW["webhook Worker<br/>(/webhooks/*)"]
  QC["Queue Consumer Workers<br/>(email / fanout / export / ai-regression)"]
  CRW["Cron Worker<br/>(daily / monthly / 5min)"]
end
D1["D1<br/>(main-db)"]
KV1["KV<br/>(session / cache / ratelimit / dedup)"]
R2["R2<br/>(exports / backups / audit-snapshots / dlq-payloads)"]
QU["Queues<br/>(email / announcement-fanout / export / ai-regression / webhook-dlq)"]
AI["Workers AI<br/>(LLM 推論)"]
SS["Secrets Store<br/>(Master Key / JWT / API キー)"]
end

subgraph Ext[" 外部サービス"]
  direction TB
  RS["Resend<br/>(メール送信 + Webhook)"]
  ST["Stripe<br/>(課金 + Webhook、顧管経由)"]
  TS["Turnstile<br/>(reCAPTCHA 代替)"]
end

subgraph Adm[" 顧客管理システム"]
  AAPI[" 内部連携 API<br/>(mTLS + 短期 JWT)"]
end

EU --> WJS
EU --> WW
AD --> PG
AD --> MW
WW --> D1
WW --> KV1
WW --> AI
MW --> D1
MW --> KV1
MW --> R2
MW --> QU
IW <--> AAPI
RS -.Webhook.-> WHW
AAPI -.IF #10 転送.-> WHW
WHW --> QU
QC --> RS
QC --> D1
QC --> R2
CRW --> D1
CRW --> QU
MW -.IF #9 / #11 送信.-> AAPI

```

1.3.2 2.2 コンポーネント詳細責務

#	コンポーネント	デプロイ単位	主責務	主使用バインディング
1	pages-admin	Cloudflare Pages	管理画面 SPA (SCR-001~025) 配信。静的アセット + クラウドフロントサイドルーティング。	-
2	pages-widget	Cloudflare Pages	widget.js および iframe コンテンツ配信。CSP/HSTS 設定。	-
3	pages-public	Cloudflare Pages	SCR-027 等の公開ページ配信。Turnstile 統合。	-

#	コンポーネント	デプロイ単位	主責務	主使用バインディング
4	worker-main-api	Workers	管理画面用 API (/api/v1/*)。Cookie + CSRF 認証。	D1 / KV / R2 / QU / AI / SS
5	worker-widget-api	Workers	ウィジェット用 API (/widget/v1/*)。bootstrap → session token 認証。	D1 / KV / AI
6	worker-internal-api	Workers	顧客管理連携 API (/internal/admin-integration/v1/*)。mTLS + 短期 JWT 認証。	D1 / KV / QU / SS
7	worker-webhook	Workers	外部 Webhook 受信 (Resend / Stripe 経由 / Turnstile)。署名検証 → Queue 投入。	D1 / KV / QU / R2 / SS
8	worker-queue-consumer	Workers Queue Consumer	Queue ジョブ実行 (email / fanout / export / ai-regression / dlq)。	D1 / R2 / QU / SS / 外部 API
9	worker-cron	Workers Cron Trigger	定期処理 (日次 / 月次 / 5 分間隔)。	D1 / KV / QU / SS

1.3.3 2.3 リクエストフロー

1.3.3.1 2.3.1 管理画面 API (admin)

sequenceDiagram

```

participant U as ブラウザ (admin)
participant P as pages-admin
participant W as worker-main-api
participant K as KV (session)
participant D as D1
participant A as audit_logs

U->>P: GET /
P-->>U: SPA + HTML/JS/CSS
U->>W: POST /api/v1/auth/login (Cookie 未保持)
W->>D: SELECT users WHERE email_hmac=?
W->>D: Argon2id verify
W->>K: SET session:{sid} (TTL 12h)
W-->>U: Set-Cookie: session=...; CSRF Cookie
U->>W: GET /api/v1/inquiries (Cookie + X-CSRF-Token)
W->>K: GET session:{sid}
W->>D: SELECT inquiries WHERE contract_owner_user_id=? AND ...
W->>A: INSERT audit_logs (read 操作は省略可)
W-->>U: 200 OK + JSON

```

1.3.3.2 2.3.2 ウィジェット API (end_user)

sequenceDiagram

```

participant E as エンドユーザー
participant W as widget.js (iframe 内)
participant API as worker-widget-api
participant K as KV (allowed_domains / threshold / session)
participant D as D1
participant AI as Workers AI

E->>W: ページ表示時にロード
W->>API: POST /widget/v1/bootstrap (pk_live_xxx, Origin)
API->>K: GET allowed_domains:{project_id}
API->>D: SELECT projects (キャッシュミス時)
API->>K: SET widget-session:{sid} (TTL 1h)
API-->>W: { session_token, project_config }
E->>W: 質問入力 → 送信
W->>API: POST /widget/v1/ask { question, session_token }
API->>D: FTS5 検索 (faq_search_fts)
API->>AI: 推論 (質問 + FAQ 抜粋)

```

```
API-->>K: GET ai_threshold:{contract_owner_user_id}:{project_id}
API-->>D: INSERT question_logs (metering_billable=...)
API-->>W: { type: "answered" | "unanswered" | "error", ... }
```

1.3.3.3 2.3.3 内部連携 API (顧客管理 → メイン)

```
sequenceDiagram
    participant ADM as 顧客管理 Worker
    participant IW as worker-internal-api
    participant K as KV (idempotency / ratelimit)
    participant D as D1
    participant Q as Queue

    ADM->>IW: POST /internal/admin-integration/v1/owner/forced-logout<br/>(mTLS + JWT + Idempoten
    IW->>IW: mTLS 証明書検証
    IW->>IW: JWT 検証 (HS256 / exp 5min / aud check)
    IW->>K: GET idempotency:{key}
    alt 既存
        IW-->>ADM: 200 OK (キャッシュ結果)
    else 新規
        IW->>D: UPDATE users SET force_logout_at = ?
        IW->>Q: SEND session-invalidation-queue
        IW->>K: SET idempotency:{key} = result (TTL 24h)
        IW-->>ADM: 201 Created
    end
```

1.3.4 2.4 環境構成

環境	用途	データ	承認
dev	開発者個人環境	テストデータのみ	なし
staging	統合テスト・MVP リリース前検証	マスキング済み本番相当	1名承認
prod	本番運用	実データ	2名承認 (NFR-805、AC-018)

1.3.4.1 2.4.1 wrangler.toml バインディング一覧 (prod 例)

```
# wrangler.toml (worker-main-api)
name = "main-api"
main = "src/index.ts"
compatibility_date = "2026-05-12"

[[d1_databases]]
binding = "DB"
database_name = "main-db-prod"
database_id = "<UUID>"

[[kv_namespaces]]
binding = "KV_SESSION"
id = "<UUID>"

[[kv_namespaces]]
binding = "KV_CACHE"
id = "<UUID>"

[[kv_namespaces]]
binding = "KV_RATELIMIT"
id = "<UUID>"

[[kv_namespaces]]
binding = "KV_DEDUP"
id = "<UUID>"
```

```
[[r2_buckets]]
binding = "R2_EXPORTS"
bucket_name = "main-exports-prod"

[[r2_buckets]]
binding = "R2_AUDIT"
bucket_name = "main-audit-prod"

[[r2_buckets]]
binding = "R2_DLQ"
bucket_name = "main-dlq-prod"

[[queues.producers]]
binding = "Q_EMAIL"
queue = "email-queue-prod"

[[queues.producers]]
binding = "Q_FANOUT"
queue = "announcement-fanout-prod"

[[queues.producers]]
binding = "Q_EXPORT"
queue = "export-queue-prod"

[ai]
binding = "AI"

[vars]
ENV = "prod"
JWT_AUD = "main.example.com"
JWT_ISS_EXPECTED = "admin.example.com"
```

1.3.4.2 2.4.2 マスキング方針 (staging)

- メールアドレス: `user{N}@example.com` 置換
- 電話番号: `0900-0000-{4 桁ハッシュ}` 置換
- 氏名: テストユーザー `{N}` 置換
- チャット本文: 長さを保ったランダム文字列に置換 (FAQ 本文は維持)

1.3.5 2.5 デプロイ単位

デプロイ単位	リポジトリ内パス (想定)	デプロイ方式	依存
pages-admin	app/admin/	Cloudflare Pages (GitHub Actions)	-
pages-widget	app/widget/	Cloudflare Pages	-
pages-public	app/public/	Cloudflare Pages	-
worker-main-api	app/workers/main-api/	wrangler deploy	D1 マイグレーション完了
worker-widget-api	app/workers/widget-api/	wrangler deploy	同上
worker-internal-api	app/workers/internal-api/	wrangler deploy	同上
worker-webhook	app/workers/webhook/	wrangler deploy	同上
worker-queue-consumer	app/workers/queue-consumer/	wrangler deploy	Queue 作成完了
worker-cron	app/workers/cron/	wrangler deploy	同上

CI/CD は GitHub Actions を使い、以下の段階を踏む。

1. PR: `vitest + tsc --noEmit + eslint`
2. main マージ: dev 環境へ自動デプロイ
3. release タグ: staging 環境へ自動デプロイ (1 名承認)

1.4 3. ディレクトリ構成・モジュール構成

1.4.1 3.1 リポジトリレイアウト

実装リポジトリ（本設計リポジトリとは別、想定パス）は以下のモノレポ構成を採用する。

faq-saas/	# 実装リポジトリ
├─ app/	
│ └─ admin/	# 管理画面 SPA (React + Vite, pages-admin)
│ └─ src/	
│ └─ pages/	# SCR-001..025 のページコンポーネント
│ └─ components/	# 共通 UI 部品 (16 部品)
│ └─ hooks/	
│ └─ lib/	# API クライアント / バリデーション / i18n
│ └─ routes.ts	# ルーティング定義
│ └─ main.tsx	
│ └─ public/	
│ └─ package.json	
│ └─ vite.config.ts	
├─ widget/	# widget.js + iframe コンテンツ (pages-widget)
├─ public/	# SCR-027 等 (pages-public)
├─ workers/	# Cloudflare Workers 群
│ └─ main-api/	# /api/v1/*
│ └─ widget-api/	# /widget/v1/*
│ └─ internal-api/	# /internal/admin-integration/v1/*
│ └─ webhook/	# /webhooks/*
│ └─ queue-consumer/	# Queue Consumer
│ └─ cron/	# Cron Trigger
└─ shared/	# 全 Worker 共通の型・ロジック
├─ migrations/	# D1 マイグレーション (forward-only)
├─ scripts/	
├─ tests/	
│ └─ unit/	
│ └─ integration/	
│ └─ e2e/	
│ └─ load/	# k6
│ └─ isolation/	# オーナー境界によるデータ分離検証
├─ .github/	
│ └─ workflows/	
├─ package.json	# ワークスペース管理 (pnpm workspace)
├─ pnpm-workspace.yaml	
├─ tsconfig.base.json	
└─ README.md	

1.4.2 3.2 レイヤ構成

各 Worker 内では以下の 5 層構成を採用する。依存方向は上から下のみを許可する（ヘキサゴナルアーキテクチャ）。

層	パス	責務	依存先
プレゼンテーション	routes/	Hono のルート定義、リクエスト/レスポンスの Zod 検証、HTTP ステータス決定	handlers
ユースケース	handlers	業務フロー（認可チェック → ドメインロジック → 永続化 → 通知 Queue 投入 → 監査ログ）	domain, repository, adapter, middleware
ドメイン	domain/	状態遷移ガード、ビジネスルール、純関数	(shared のみ)

層	パス	責務	依存先
アダプタ	adapter/	外部 API クライアント (Resend / Stripe / 顧客管理)、Queue 投入、KV/R2 アクセス	(shared のみ)
リポジトリ	repository/	DB SQL 実行、トランザクション	(shared のみ)
ミドルウェア	middleware/	認証 / CSRF / 認可 / 監査 / レート制限	repository, adapter, shared
ライブラリ	lib/	Worker 固有のユーティリティ	shared
共通	shared/	全 Worker 共通の型・スキーマ・純関数	(外部 npm のみ)

1.4.3 3.3 主要モジュール

1.4.3.1 3.3.1 worker-main-api モジュール構成

```
src/
├── index.ts                # Hono app entry + 全ミドルウェア配線
├── routes/
│   ├── auth.ts            # /auth/*
│   ├── admin-users.ts     # /admin-users/*
│   ├── projects.ts        # /projects/*
│   ├── faqs.ts            # /faqs/*
│   ├── inquiries.ts       # /inquiries/*
│   ├── chat-rooms.ts      # /chat-rooms/*
│   ├── usage.ts           # /usage
│   ├── billing.ts         # /billing/*
│   ├── data.ts            # /data/*
│   ├── withdrawal.ts      # /withdrawal/*
│   ├── announcements.ts   # /me/announcements/*
│   ├── notification-prefs.ts # /me/notification-preferences
│   ├── terms.ts           # /me/terms*
│   └── email-verification.ts # /me/email-verification/*
├── handlers/              # routes と 1:1 対応
├── domain/
│   ├── faq-status.ts
│   ├── inquiry-status.ts
│   ├── chat-room-status.ts
│   ├── notification-status.ts
│   ├── contract-status.ts
│   ├── pii-scrubber.ts
│   ├── post-check.ts
│   ├── ai-threshold.ts
│   ├── inquiry-code.ts
│   └── ...
├── repository/            # 23 テーブル × 1 ファイル
├── adapter/
│   ├── email-provider.ts
│   ├── resend-email-provider.ts
│   ├── answer-provider.ts
│   ├── workers-ai-answer-provider.ts
│   ├── admin-integration-client.ts
│   └── stripe-client.ts
├── middleware/
│   ├── authenticate.ts
│   ├── csrf.ts
│   ├── authorize.ts
│   ├── require-active-contract.ts
│   ├── require-terms-agreement.ts
│   └── require-reauth.ts
```



```

|   |— rate-limit.ts
|   |— audit.ts
|   |— error-handler.ts
|— lib/
|   |— kv.ts
|   |— d1.ts
|   |— queue.ts
|   |— ulid.ts
|   |— argon2id.ts
|   |— token.ts
|   |— encrypt.ts
|   |— hmac.ts
|   |— audit-hash.ts
|   |— ip-mask.ts
|   |— ip-allowlist.ts
|   |— logger.ts
|— types.ts

```

1.4.3.2 3.3.2 worker-widget-api モジュール構成

```

src/
|— index.ts
|— routes/
|   |— bootstrap.ts
|   |— ask.ts
|   |— inquiries.ts
|   |— chat-rooms.ts
|   |— reentry.ts          # SCR-027 連携
|— handlers/
|— repository/
|— adapter/
|— middleware/
|   |— verify-widget-key.ts
|   |— widget-session.ts
|   |— rate-limit.ts
|   |— audit.ts
|— lib/

```

1.4.3.3 3.3.3 worker-internal-api モジュール構成

```

src/
|— index.ts
|— routes/
|   |— owner.ts           # IF #1 (suspend/resume) / IF #2 (forced-logout)
|   |— restore.ts        # IF #4
|   |— rate-limit.ts     # IF #5
|   |— threshold.ts      # IF #6
|   |— announcement.ts   # IF #7
|   |— metrics.ts        # IF #8
|   |— billing-webhook.ts # IF #10
|   |— operator-operation.ts # IF #12
|   |— ai-regression.ts   # v2.1 新規
|— middleware/
|   |— verify-mtls.ts
|   |— verify-jwt.ts
|   |— idempotency.ts
|   |— audit.ts

```

└ ...

1.4.3.4 3.3.4 worker-webhook モジュール構成

```
src/
├─ index.ts
├─ routes/
│  └─ resend.ts
│  └─ stripe.ts          # 実質 IF #10 経由
├─ handlers/
│  └─ resend/            # 8 イベント種別
│     ├── delivered.ts
│     ├── bounced.ts
│     ├── complained.ts
│     ├── delayed.ts
│     ├── opened.ts
│     ├── clicked.ts
│     ├── failed.ts
│     └── suppressed.ts
│  └─ stripe/            # 9 イベント種別 (IF #10)
│     ├── invoice-paid.ts
│     ├── invoice-payment-failed.ts
│     ├── customer-subscription-created.ts
│     ├── customer-subscription-updated.ts
│     ├── customer-subscription-deleted.ts
│     ├── customer-subscription-trial-will-end.ts
│     ├── charge-refunded.ts
│     ├── charge-dispute-created.ts
│     └── customer-tax-id-updated.ts
├─ middleware/
│  ├── verify-resend-signature.ts
│  ├── verify-stripe-signature.ts
│  └── idempotency.ts
└ ...
```

1.4.3.5 3.3.5 worker-queue-consumer モジュール構成

```
src/
├─ index.ts              # Queue Consumer 共通エントリ
├─ consumers/
│  ├── email.ts          # Resend 送信
│  ├── fanout.ts         # お知らせ fan-out
│  ├── export.ts         # データエクスポート生成
│  ├── ai-regression.ts  # AI 回帰テスト実行
│  └── dlq.ts            # DLQ 退避 (R2 退避 + 監視通知)
├─ lib/
│  └─ retry.ts           # 指数バックオフ + 最大 3 回
└ ...
```

1.4.3.6 3.3.6 worker-cron モジュール構成

```
src/
├─ index.ts              # scheduled() ハンドラ + cron 分岐
├─ jobs/
│  ├── monthly-aggregate.ts    # UTC 15:00 (月末日)
│  ├── monthly-finalize.ts     # JST 02:00 (月初 1 日)
│  ├── trial-end-check.ts      # JST 00:00 (毎日)
│  ├── open-inquiry-retention-notice.ts # JST 09:00 (毎日)
│  └── open-inquiry-retention.ts # JST 02:00 (毎日)
└ ...
```

```

├── auto-close-evaluation.ts    # */5 * * * * (5 分間隔)
├── audit-chain-verify.ts      # JST 03:00 (日次全件再計算)
├── tombstone-batch.ts        # JST 04:00 (毎日)
├── retention-cleanup.ts       # JST 05:00 (毎日)
├── d1-capacity-check.ts       # 0 * * * * (毎時)
└── lib/

```

1.4.4 3.4 共通ライブラリ

app/shared/src/ 配下に配置し、全 Worker から @faq-saas/shared として import する。

モジュール	主エクスポート	用途
schemas/	inquirySchema, faqSchema, askRequestSchema, ...	Zod スキーマ (API リクエスト / レスポンス / DB レコード)
constants/status.ts	INQUIRY_STATUS, FAQ_STATUS, CHAT_ROOM_STATUS, REMINDER_STATE, NOTIFICATION_STATUS, TENANT_STATUS, DELETION_LOCAL_STATUS	状態の文字列定数
constants/error-codes.ts	ErrorCode enum	全 40+ エラーコード
constants/notification-types.ts	NOTIFICATION_TYPES (14 種)	メール送信契機
constants/audit-actions.ts	AUDIT_ACTIONS (9 カテゴリ網羅)	監査ログ action コード
domain/inquiry-status-transition.ts	canTransition(), nextStatus()	状態遷移ガード (純関数)
domain/faq-status-transition.ts	同上	FAQ
domain/chat-room-status-transition.ts	同上	部屋
lib/ulid.ts	generateUlid()	ULID v7 風生成
lib/hmac.ts	hmacSha256(key, data)	HMAC 計算
lib/encrypt.ts	aesGcmEncrypt(), aesGcmDecrypt(), deriveOwnerKey()	AES-256-GCM + HKDF
lib/argon2id.ts	hashPassword(), verifyPassword()	Argon2id ラッパ
lib/token.ts	generateToken(), verifyToken()	HMAC-SHA256 トークン
lib/audit-hash.ts	computeChainHash()	ハッシュチェーン
lib/ip-mask.ts	maskIp(ip)	IPv4/IPv6 マスク
lib/inquiry-code.ts	generateInquiryCode()	INQ-YYYYMMDD-XXXXXXXX
lib/logger.ts	Logger	structured logging
i18n/ja.ts	messages	日本語メッセージカタログ

1.4.5 3.5 設定ファイル

1.4.5.1 3.5.1 wrangler.toml の管理

- 環境別ファイル: wrangler.toml (dev デフォルト), wrangler.staging.toml, wrangler.prod.toml
- デプロイ時に --config で切替
- バインディング ID は環境ごとに異なる (KV namespace / D1 database / R2 bucket / Queue は環境別に作成)

1.4.5.2 3.5.2 シークレット管理 すべての機密値は Cloudflare Secrets Store (wrangler secret put) に格納し、コード・wrangler.toml に直書きしない。

シークレット名	用途	ローテーション
MASTER_KEY	オーナー派生 の HKDF 元 / 暗号化列 / トークン HMAC	年次
JWT_HS256_KEY	連携 IF JWT 署名	年次

シークレット名	用途	ローテーション
RESEND_API_KEY	Resend API 認証	必要時
TURNSTILE_SECRET	Turnstile 検証	必要時
ADMIN_INTEGRATION_M...	顧客送信時のクライアント証明書	年次
ADMIN_INTEGRATION_M...	OS上の秘密	年次

1.5 4. 共通処理方針

1.5.1 4.1 エラー設計（参照のみ）

エラー分類 11 種、エラー ID 体系（E-*）、ステータスコード対応、異常系共通方針の全項目は [../02_基本設計/05_エラー設計.md](#) を正本とする。

1.5.2 4.2 セキュリティ詳細設計（参照のみ）

具体的なセキュリティ施策・PII 暗号化・Web 脆弱性対策・管理・監査ログ詳細・不正利用検知・メール配信信頼性の全項目は [../02_基本設計/09_セキュリティ設計.md](#) を正本とする。

1.5.3 4.3 ログ設計概要

種別	保存先	保持期間	PII
監査ログ	audit_logs (D1)	1/5/7 年	マスク済み
エラーログ	error_logs (D1)	180 日	マスク済み
通知ログ	notification_logs (D1)	1 年	HMAC のみ
アクセスログ	Cloudflare Logpush → R2	180 日	IP マスク
連携 IF ログ	integration_logs (D1)	90 日	-

- ・ 監査・エラー・通知ログは D1 内（クエリ可能）
- ・ アクセスログは R2（圧縮、長期保存）
- ・ すべて Cloudflare Logpush / R2 / Analytics Engine の運用ログ基盤に集約
- ・ 構造化ログには **trace_id** (ULID) と **cf_ray** (Cloudflare) を必須で出力し、CDN 側ログとアプリ層ログを突合可能とする

1.5.4 4.4 監視・アラート

監視・アラート運用の詳細は [../04_運用設計/index.md](#) の「メインシステム 監視・アラート詳細」に集約する。本書では実装・テスト設計との接点のみを扱う。

1.5.5 4.5 実装ガイドライン

1.5.5.1 4.5.1 TypeScript / Hono / Cloudflare Workers の規約

- ・ **strict mode**: "strict": true 必須。any 禁止（unknown を使う）
- ・ **import 順**: 外部ライブラリ → @faq-saas/shared → 相対 import
- ・ **エクスポート**: named export を基本とし、default export は React コンポーネント等のみ
- ・ **非同期**: Promise チェーンより **async/await**
- ・ **エラー throw**: **HTTPException** (Hono) または **Error** サブクラス。文字列 throw 禁止

1.5.5.2 4.5.2 命名規則

対象	ルール	例
ファイル	kebab-case	inquiry-status.ts, widget-key-rotate.ts
クラス	PascalCase	WorkersAIAnswerProvider
関数 / 変数	camelCase	generateInquiryCode, contractOwnerUserId
定数	UPPER_SNAKE_CASE	MAX_RETRY, DEFAULT_LIMITS
型	PascalCase	Inquiry, AnswerInput

対象	ルール	例
Zod スキーマ	xxxSchema	loginSchema, createFaqSchema
DB テーブル / カラム	snake_case	テーブル設計
API パス	kebab-case	/chat-rooms/{id}/messages

1.5.5.3 4.5.3 エラーハンドリング規約

- `handlers/` 層で `throw new HTTPException(...)` を使い、`errorHandler` ミドルウェアで一括変換
- リトライ対象 / 非対象を明示するため、`SystemError`, `BusinessError` の 2 種カスタム例外を派生
- `Result` 型は不使用 (`throw` ベースに統一)

1.5.5.4 4.5.4 セキュアコーディング (OWASP Top 10)

OWASP 項目	対応
A01 Broken Access Control	認可ロジック、 <code>requireProject</code> / <code>requireInquiry</code> / 境界チェック
A02 Cryptographic Failures	AES-256-GCM、HMAC、HKDF
A03 Injection	Zod 検証、D1 Prepared Statements 必須、CSP
A04 Insecure Design	状態遷移ガード、3 層公開禁止、再認証
A05 Security Misconfiguration	HTTP セキュリティヘッダ、Secrets Store
A06 Vulnerable Components	<code>pnpm audit</code> を CI に組み込み
A07 Identification & Authentication	Argon2id、Turnstile、ロックアウト
A08 Software & Data Integrity	監査ログハッシュチェーン、forward-only マイグレーション
A09 Logging Failures	構造化ログ、監査ログ、PII マスク
A10 SSRF	外部 fetch 先を allowlist 化 (Resend / Stripe API のみ)

1.6 5. 詳細設計ファイル一覧

ファイル	ドメイン	対応 FR	主関連章
DD01_アカウント・ユーザー管理・ユーザー管理.md	アカウント・ユーザー管理 / フロント URL ルーティング / Onboarding Checklist	FR-001~022 / FR-325~327	§ 4 / § 6 SCR-001~003, 017, 026 / § 7
DD02_プロジェクト・FAQ 管理.md	プロジェクト・FAQ 管理	FR-040~048 / FR-100~103, FR-105, FR-106	§ 6 SCR-010~012 / § 7 / § 9
DD03_AI 回答パイプライン.md	AI 回答パイプライン	FR-050~060	§ 10.1
DD04_AI しきい値 3 階層適用.md	AI しきい値 3 階層適用	FR-340 / FR-341	§ 10.2
DD05_inquiry_code 採番・未解決質問.md	inquiry_code 採番・未解決質問	FR-070~079 / FR-072	§ 10.3 / § 6 SCR-011 / § 7
DD06_個別チャット.md	個別チャット	FR-080~091	§ 6 SCR-013 / § 7
DD07_通知ロジック.md	通知ロジック	FR-140~149	§ 10.4
DD08_トークン発行・検証.md	トークン発行・検証	FR-003 / 004 / 006 / 083 / 084	§ 10.5
DD09_認可ヘルパ.md	認可ヘルパ	NFR-301 / FR-007	§ 10.6
DD10_監査ログ書込・完全性検証.md	監査ログ書込・完全性検証	NFR-601 / NFR-602	§ 10.7 / § 10.8 / § 15.2
DD11_暗号化・管理.md	暗号化・管理	NFR-301 / NFR-401	§ 10.9

ファイル	ドメイン	対応 FR	主関連章
DD12_ 利用量計測・課金.md	利用量計測・課金	FR-120～127 / FR-139	§ 6 SCR-015 プロジェクト視点 / SCR-026 オーナー視点 / 課金部は契約 WS 配置 (基本設計 § 4.4.3 / § 5.10 が正本)
DD13_ ウィジェット配信.md	ウィジェット配信	FR-150～156 / FR-031 / FR-083 / 084	§ 6 SCR-014(プロジェクト WS 配置・公開キーはプロジェクト単位) / SCR-027(基本設計が正本)
DD14_ バッチ・非同期処理.md	バッチ・非同期処理	FR-148 / FR-191	§ 10.11 / § 14
DD15_ アクセシビリティ・国際化.md	アクセシビリティ・国際化	NFR-401 / WCAG 2.1 AA	§ 13.4 / § 13.5

1.7 6. 実装順序

実装順序は依存関係を考慮し、横断的基盤機能から積み上げ、ドメイン機能を後段で組み立てる。

1. **DD11 暗号化・鍵管理** —MASTER_KEY / HKDF / AES-256-GCM の基盤が他すべての依存元
2. **DD08 トークン発行・検証** —メール検証・パスワードリセット・再入室で利用
3. **DD09 認可ヘルパ** —requireTenant / requireProjectRole / requireInquiry / requireOwner は全 API 共通
4. **DD10 監査ログ書込・完全性検証** —全業務処理で writeAudit を必須呼出
5. **DD01 アカウント・ユーザー管理** —認証・オーナー / プロジェクト管理者 / メンバーの 3 ロール
6. **DD02 プロジェクト・FAQ 管理** —プロジェクト / FAQ CRUD + FTS5
7. **DD03 AI 回答パイプライン** —Workers AI / PostCheck / PII scrubber
8. **DD04 AI しきい値 3 階層適用** —KV / 永続キャッシュ / フォールバック
9. **DD13 ウィジェット配信** —bootstrap / ask / SCR-027 再入室
10. **DD05 inquiry_code 採番・未解決質問** —INQ-YYYYMMDD-XXXXXXXXXX
11. **DD06 個別チャット** —チャット部屋・メッセージ
12. **DD07 通知ロジック** —Resend / Queue / Webhook
13. **DD12 利用量計測・課金** —月次集計 / Stripe 連動
14. **DD14 バッチ・非同期処理** —cron / Queue / DLQ 全体
15. **DD15 アクセシビリティ・国際化** —WCAG / i18n (横断的、リリース前ゲート)

1.8 7. 非機能設計サマリ

1.8.1 7.1 性能設計 (§13.1)

- ・公開ウィジェット POST /widget/v1/bootstrap p95 < 200ms
- ・POST /widget/v1/ask (FAQ ヒット) p95 < 1000ms / (miss) < 500ms
- ・AI 推論 p95 < 2500ms (タイムアウト 5000ms)
- ・管理画面一覧 p95 ☒ 800ms (NFR-105)
- ・AI 推論タイムアウト時は **unanswered** + 管理者ユーザー誘導 モードでウィジェット動作継続

1.8.2 7.2 可用性設計 (§13.2)

指標	SLO	エラーバジェット (月)
公開ウィジェット可用性	99.9%	43.2 分
管理画面可用性	99.5%	216 分
公開 API 5xx 率	< 0.5%	-
通知配信失敗率	< 1%	-
AI 推論 p95	< 2.5s	月 5% 超過まで

- ・顧客管理障害時もウィジェット応答 / 既存チャット返信 / FAQ 検索編集 / 既存ログインは継続
- ・**announcement.kind = critical** は障害時に Resend 直送フォールバック経路を持つ

1.8.3 7.3 スケーラビリティ (§13.3)

- ・ D1 シャーディング判定閾値: アクティブ契約数 150 / D1 容量 80% (8 GB) / p95 SELECT > 200ms 連続
- ・ AI 推論並行数: グローバル 200 / オーナー 10 (KV カウンタ)

1.8.4 7.4 アクセシビリティ・国際化

詳細は DD15 参照。WCAG 2.1 AA 準拠、i18n 基盤は `react-i18next + app/shared/src/i18n/ja.ts`。

1.8.5 7.5 バックアップ・復旧 (§13.6)

復旧経過時間	採用バックアップ	RTO	RPO
0~30 日	D1 Time Travel (PITR)	☑ 30 分	☑ 1 分
30 日超~12 週	R2 週次バックアップ	☑ 2 時間	週次
12 週超~12 か月	R2 月次バックアップ	☑ 4 時間	月次
12 か月超	R2 監査アーカイブ (5y/7y)	☑ 1 営業日	法令保持内

- ・ AES-256-GCM **BACKUP_KEY**、年次ローテーション + 60 日 dual-decrypt 期間
- ・ 復元時整合性検証 4 項目 (スキーマバージョン / 行件数 ±5% / チェックサム / KPI 差分)

1.8.6 7.6 データ保持・削除 (§13.7)

データ保持期間は **システム固定値** (NFR-702、UI 上の変更不可)。SCR-028 アカウント設定にデータ保持期間設定 UI は存在しない ([01_画面設計.md § 5.20b](#) 参照)。

データ種別	保持期間	起点
FAQ	無期限	-
question_logs / inquiries / chat / 通知 / inbox	1 年	created_at / closed_at
監査ログ (general / billing / operator_high_priv)	1y / 7y / 5y	created_at
エラーログ	180 日	occurred_at

調整制約は [03_テーブル設計.md § 10](#) を正本。短縮は法令・契約義務範囲内でサポート窓口経由運営者対応、長期化は将来要件 (FUT 配下) で再検討。

1.8.7 7.7 コンプライアンス (§13.X)

- ・ 個人情報保護法 / GDPR / 電子帳簿保存法 / 特定電子メール法 / ISO/IEC 27017 を適用
- ・ PCI-DSS はスコープ外宣言 (SAQ A 相当、Stripe Elements 完全委託)

1.8.8 7.8 インフラコスト構造 (§13.Y)

Cloudflare Workers / Workers AI / D1 / KV / R2 / Queues / Browser Rendering / Pages / Logpush の課金単位と発生源を四半期で試算管理。[docs/operations/cost-estimate/YYYY-Q.xlsx](#) で記録。

1.9 8. リリース計画

移行・リリース運用の詳細は [./04_運用設計/index.md](#) の「メインシステム 移行・リリース詳細」に移管した。本書では実装成果物との対応関係のみを扱う。

1.9.1 8.1 CI/CD パイプライン

1. PR: `vitest + tsc --noEmit + eslint`
2. main マージ: dev 環境へ自動デプロイ
3. release タグ: staging 環境へ自動デプロイ (1 名承認)
4. prod タグ: prod 環境へデプロイ (**2 名承認**、GitHub Environment Protection)

1.9.2 8.2 ペネトレーション

MVP リリース前に外部委託で OWASP Top 10 + Cloudflare 環境特有のチェックを実施。レポートは社内共有 + 是正計画。

1.9.3 8.3 リリースゲート

- AI 品質回帰テスト合格（旧モデル正答一致率 ☒ 90% / 信頼度差 ± 0.10 / 回答可能率劣化 -5pt 以内）
 - 監査ハッシュチェーン整合性（直近 7 日ゼロ違反）
 - 復旧訓練成功記録（直近 1 回以上、4 時間以内 + 整合性検証 4 項目合格）
 - AC マッピング Test ID 未紐付け 0 件
-

1.10 9. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
TBD-001	NFR-101~104 の p95 数値・測定ツールの最終選定	高	未確定（PO + SRE）
TBD-002	復旧訓練の所要時間ログと記録保管場所	中	未確定（SRE）
TBD-003	i18n-lint 等の翻訳カタログ検証ツール選定	中	未確定（フロントリード）
TBD-004	Workers AI Region 指定 SDK オプション名（公式仕様追随）	中	未確定（バックエンドリード）
完了	11 ドメイン分割（DD01~DD15）	-	完了